

Lesson 6: SQL Injection

Marcin Kurzyna

Lecture Goals

This lecture introduces:

- What SQL Injection is,
- How databases process SQL queries,
- Unsafe query construction,
- Authentication bypass attacks,
- Consequences of SQL Injection,
- Secure coding practices,
- Parameterized queries in Python.

1 What Is SQL Injection?

SQL Injection is a security vulnerability where user input is interpreted as SQL code. Attackers can:

- Read database contents,
- Modify data,
- Delete records,
- Bypass authentication.

2 How SQL Queries Work

Example SQL query:

```
SELECT * FROM users
WHERE username = 'admin'
AND password = 'secret123';
```

The database executes the query exactly as written.

3 Unsafe User Input

Suppose a login system builds SQL queries like this:

```
username = input("Username: ")
password = input("Password: ")

query = "SELECT * FROM users WHERE username = '" \
        + username + "' AND password = '" \
        + password + "'"
```

Problem: User input becomes part of SQL code.

4 Example Attack

Attacker enters:

Username: admin
Password: ' OR '1'='1

Generated query:

```
SELECT * FROM users
WHERE username = 'admin'
AND password = '' OR '1'='1';
```

Because:

'1'='1'

is always true, authentication may be bypassed.

5 Why This Happens

The application:

- Trusts user input,
- Concatenates strings directly,
- Does not separate data from commands.

6 Consequences of SQL Injection

Possible attacks include:

- Login bypass,
- Data theft,
- Data deletion,
- Changing user permissions,
- Complete database compromise.

7 Example: Reading All Users

Unsafe query:

```
query = "SELECT * FROM users WHERE id = " + user_id
```

Attacker input:

```
1 OR 1=1
```

Final query:

```
SELECT * FROM users WHERE id = 1 OR 1=1
```

Result:

- Entire table may be returned.

8 Example: Deleting Data

Attacker input:

```
1; DROP TABLE users;
```

Possible result:

```
SELECT * FROM users WHERE id = 1;  
DROP TABLE users;
```

Danger: Database tables may be deleted.

9 Basic Protection Methods

Never:

- Build SQL queries using string concatenation,
- Trust user input.

Always:

- Use parameterized queries,
- Validate input,
- Use least-privilege database accounts.

10 Part 2: Practical Implementation in Python

11 Unsafe SQLite Example

```
import sqlite3

conn = sqlite3.connect("users.db")
cursor = conn.cursor()

username = input("Username: ")
password = input("Password: ")

query = "SELECT * FROM users WHERE username='" \
        + username + "' AND password='" \
        + password + "'"

cursor.execute(query)

result = cursor.fetchone()

if result:
    print("Access granted")
else:
    print("Access denied")
```

12 Safe Parameterized Query

```
import sqlite3

conn = sqlite3.connect("users.db")
cursor = conn.cursor()

username = input("Username: ")
password = input("Password: ")

query = "SELECT * FROM users WHERE username=? AND password=?"

cursor.execute(query, (username, password))

result = cursor.fetchone()

if result:
    print("Access granted")
else:
    print("Access denied")
```

13 Why Parameterized Queries Are Safe

The database:

- Separates SQL code from user data,
- Treats input only as data,
- Prevents injected SQL commands.

14 Input Validation

Example:

```
user_id = input("Enter ID: ")

if user_id.isdigit():
    print("Valid ID")
else:
    print("Invalid input")
```

15 Principle of Least Privilege

Database accounts should:

- Have minimal permissions,
- Avoid administrator rights,
- Restrict dangerous operations.

16 Comparison

Method	Secure	Recommended
String concatenation	No	Never
Input filtering only	Partial	Not enough
Parameterized queries	Yes	Recommended
ORM frameworks	Yes	Recommended

Summary

In this lecture, we covered:

- What SQL Injection is,
- How unsafe SQL queries are created,
- Authentication bypass attacks,
- Risks of SQL Injection,

- Parameterized queries,
- Secure database programming practices.

17 Exercises

1. Write an unsafe SQL login query.
2. Perform a simple SQL Injection attack.
3. Explain why the attack works.
4. Rewrite the query using parameterized statements.
5. Add input validation for numeric IDs.
6. Explain the principle of least privilege.